Cohort 0 Final Report

1. TEAM INFO

- **Team name:** Vikranth Reddimasu's Team-tiok
- **Member:** Vikranth Reddimasu (solo)
- **Agent name:** Arya (after Aryabhata)

2. WHAT I BUILT

Arya is intentionally minimal and entirely prompt-driven, running on the Goose harness with Minimax M2.5 via OpenRouter. No MCP servers, nor external tools beyond what the harness provides (shell and Python). The prompt encodes a strict workflow: SEARCH → EXTRACT → WRITE preliminary answer within 3-5 tool calls → COMPUTE → STOP.

Write-early rule: Requiring a preliminary answer before any further computation eliminated most of the empty answer.txt failures. The prompt also carries pre-written formulas for every computation type encountered (CAGR, moving averages, quartiles, Fisher kurtosis, CPI adjustment, Theil index, geometric mean), explicit pipe-column parsing instructions, fiscal vs. calendar year timing rules, and a 10-rule completion checklist.

Why no MCP servers: The cost penalty in the scoring formula is steep, and the corpus is pre-parsed into grep-searchable text files, making a grep→sed→python3 pipeline sufficient. Nine skill files were also built but never loaded at runtime for most of the submissions with goose. The 191.0 peak score was achieved on the system prompt alone.

Top submissions.

Metric	Best Score Run	Highest Accuracy Run
Date	Apr 5, 2026	Apr 3, 2026
Harness	Goose	OpenHands-SDK
Score	191.0	188.0
Success rate	70.7%	72.0%
Cost	$0.01	$0.07
Avg time/task	186.01 s	144.39 s

The 72.0% success rate is the highest accuracy ever recorded on the OfficeQA benchmark, surpassing all prior frontier agent results. This run scored lower overall (188.0 vs. 191.0) because OpenHands-SDK incurred $0.07 cost vs. Goose's $0.01. Another team matched my second best accuracy of 70.7% but scored ~1 point higher due to a 0.3 s/task latency advantage.

3. HOW I WORKED

Local model: Local runs used Minimax M2.7 rather than M2.5: 2× max output tokens, slightly more context, marginally higher cost, better throughput. This produced higher-quality traces to analyze even though the server always runs M2.5.

Version evolution: Three inflection points shaped the final design. (1) v0.1→v1.0: Discovered early submissions contained leaked benchmark answers in skill example files, so I cleaned and replaced them with fictional values. (2) v4.0: Shorter prompts cost more, input tokens hit ~96% cache hit rate (essentially free), but each extra iteration generates full-price output tokens. Stripping inline guidance pushed cost from $12 to $19 with negligible accuracy gain. (3) v8.0: Found the 6.3 KB sweet spot. Every modification after that like formatting rules, inlined skills, harness swaps mostly caused regression.

Trajectory failure patterns: Column miscounting: despite explicit split('|') instructions, the agent visually estimated positions rather than splitting programmatically. Wrong final formula: intermediate values often correct, but percent change instead of percent difference, or re-multiplying an already-percentage value. Fiscal year lag: Treasury Bulletins publish 1-3 months after the reporting period; the agent did not consistently account for this.

4. WHAT I FOUND

- **Simplicity wins:** Every content addition like formatting rules, inlined skills caused regression. M2.5 performs best with concise, direct instructions. The 6.3 KB sweet spot was found through 20+ submissions.
- **Variance dominates.** The same v8.0 code produced scores from 150 to 191, that is a 41-point spread. 5 of 20 tasks flipped between pass and fail on identical code.
- **Model choice matters:** The server always runs M2.5 regardless of arena.yaml. Available levers: system prompt, skill files (non-functional), MCP servers (similar issues).
- **Cost-accuracy tradeoff is counterintuitive:** Input caching makes longer prompts essentially free. More inline guidance reduces output tokens by solving tasks in fewer turns.

Failure pattern analysis (20-task local study, 2 identical runs):

Category	n	Root cause	Fixable?
Chart / figure questions	2	Data exists only in images, not in text corpus	No
Column extraction error	4	Visual column estimation despite explicit parsing rules	Partial
100× unit / scale error	2	% vs. ratio confusion, value already in %, agent multiplies again	Flaky
Formatting mismatch	2	Trailing newlines, spaces after commas, wrong sign	Sensitive
Non-deterministic	5	Same question yields different answers across runs	No

What worked: Write-early rule (eliminated empty-file failures); Python-only computation (eliminated mental math errors); pre-written formulas (model copies rather than invents); split('|') column indexing; lean prompt (fewer iterations, lower cost, higher score).

What did not work: Formatting constraints (more tokens, no scoring benefit); inlining skill content (bloated prompt, higher cost); removing the formula toolkit (model still needed explicit references).

Future directions: Custom MCP server for Treasury Bulletin structure-aware retrieval; multi-pass verification on high-uncertainty tasks; systematic A/B testing with sufficient submissions to overcome variance; chart question workarounds via equivalent text-table data elsewhere in the corpus.

5. FEEDBACK FOR ARENA

- **Model config clarity:** The model field in arena.yaml has no server-side effect. Documenting or removing it would prevent wasted local submissions since model selection doesn't matter server side.
- **Reduce variance:** A 41-point spread on identical code makes systematic improvement very difficult. Averaging across multiple runs, or deterministic seeding, would give participants a cleaner signal.
- **Task-level feedback:** Only the aggregate score is visible post-submission for most of the development period. Pass/fail per task without revealing expected answers would enable targeted optimization rather than blind search.

What worked well: Local trajectory inspection was invaluable for understanding failure modes unavailable from aggregate scores alone. The Discord community surfaced the most critical platform information, like the summon bug, model override behavior, harness tips. The competition format was genuinely educational, pushing deep understanding of prompt engineering, cost optimization, and agent evaluation mechanics in ways documentation alone never would. I hope both participants and the sentient team have learned a lot in this journey. Looking forward to Cohort 1.